

Project Development Continuation - GymTrack

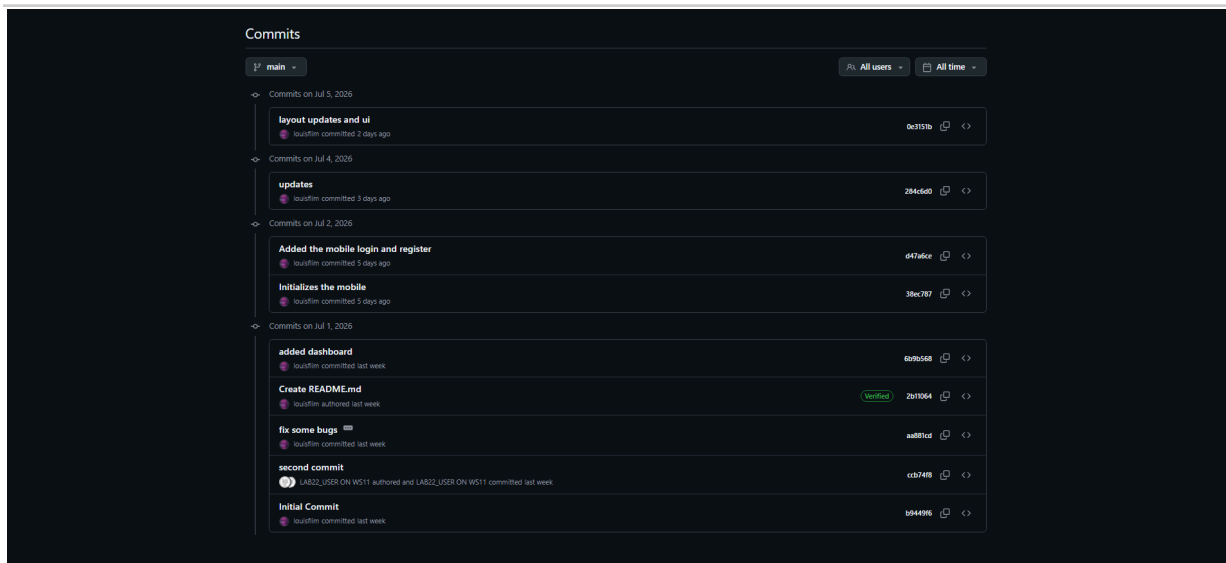
Louis Francis Lim

7/7/2026

1. GitHub Commit History

Repository: github.com/louisflim/gymtrack

Commit log: github.com/louisflim/gymtrack/commits/main



#	Commit	Date	Message	What changed
1	b9449f6	2026-07-01 07:45	Initial commit	Set up the repo and base folder structure for backend, web, and mobile.
2	ccb74f8	2026-07-01 18:00	second commit	Spring Boot scaffold with JWT security, AuthController, and the User entity; matching React login/register pages; Supabase datasource wired up.
3	aa881cd	2026-07-01 19:18	fix some bugs	Fixed the web dev-server proxy config and a couple of database connection properties that were pointing to the wrong host.
4	2b11064	2026-07-01 19:24	Create README.md	Added a README covering the SRS summary, tech stack, and how to run the project locally.
5	6b9b568	2026-07-01 19:47	added dashboard	Renamed the frontend folder to web/, added the first dashboard page shell, and dropped in the final SRS PDF under docs.
6	38ec787	2026-07-02 10:17	Initializes the mobile	Android project setup: Retrofit client, DataStore for session, Compose auth screens, nav graph, app theme.
7	d47a6ce	2026-07-02 19:05	Added the mobile login and register	Cleaned up the web auth UI, added QR/attendance controllers on the backend, added the staff-creation endpoint, and built the

#	Commit	Date	Message	What changed
				mobile dashboard with the QR scanner and member QR card.
8	284c6d0	2026-07-04 08:05	updates	Bigger batch: membership, plans, and PayMongo payments; admin dashboard KPIs; member and staff management endpoints; gym-enrollment QR flow; the web admin panels and their Android equivalents.
9	0e3151b	2026-07-05 08:20	layout updates and ui	Moved the mobile UI off Jetpack Compose and onto XML layouts with Fragments, ViewBinding, and RecyclerView adapters, plus the QR scanner dialog fragment.

Commits 8 (284c6d0) and 9 (0e3151b) each cover more than one feature. Most of the plans/payments/dashboard work came together in one long working session and I committed it as a batch instead of splitting it up feature by feature, and the mobile UI migration touched every screen at once so splitting it further wouldn't have made the history clearer. Everything else in the table is one feature per commit.

Rough mapping from feature area to the commit that introduced it:

Feature area	Commit
User authentication (backend + web)	ccb74f8
Web dashboard shell	6b9b568
Android project setup	38ec787
QR attendance + web/mobile auth UI	d47a6ce
Plans, payments, membership, admin dashboard	284c6d0
Mobile XML UI migration	0e3151b

2. Implementation Explanation Report

2.1 Login and account roles

Gym owners register as admins, members sign themselves up, and admins can create staff accounts afterward. The main thing I had to get right here was making sure a member couldn't just call the staff-creation endpoint directly and grant themselves a role they shouldn't have.

On web, `Login/index.jsx` and `Register/index.jsx` are the pages, with `RegisterForm.jsx` and `CreateStaffForm.jsx` handling the actual form logic and `api/auth.js` talking to the server. Android splits the same job between `LoginFragment` and `RegisterFragment`, backed by `AuthViewModel` and `AuthRepository`.

The request hits `AuthController`, which passes it to `AuthService` that's where the role gets checked, the password gets hashed with `BCrypt`, and the `User` row (tied to a `Gym` and a `Role`) gets saved. `JwtUtil` then issues a token. `SecurityConfig` and `JwtAuthFilter` check that token on every later request, and `axiosInstance.js` / `AuthInterceptor.kt` attach it automatically on web and mobile.

Method	Endpoint	Description
POST	<code>/api/auth/register</code>	Create an admin or member account
POST	<code>/api/auth/login</code>	Authenticate and return a JWT
POST	<code>/api/auth/staff</code>	Admin creates a staff account

Tables touched: users, gyms.

I tested this by registering as a member, logging in as admin, creating a staff account, and then trying to hit `/api/auth/staff` while logged in as a plain member that gets rejected, as it should. Wrong login credentials return a proper error message instead of a blank screen.

2.2 Member management

Once accounts exist, admins need a way to actually manage the people in their gym: search, filter, edit a profile, assign a plan. Members need the flip side of that, a view of their own status.

`MemberTable.jsx` and `MemberDashboardPanel.jsx` cover this on web, with `MemberOnboardingCard.jsx` for first-time setup and `api/members.js` doing the fetching. Android reuses `DashboardFragment` for the same screens, with `MemberAdapter` rendering the list.

`MemberController` and `MembershipController` route through `MemberService` and `MembershipService`, which read and write via `UserRepository` and `MembershipRepository`.

Method	Endpoint	Description
GET	<code>/api/members</code>	List / search / filter members (admin)
PUT	<code>/api/members/{id}</code>	Update a member's profile
POST	<code>/api/members/assign-plan</code>	Assign a subscription plan
GET	<code>/api/membership/me</code>	Member checks their own membership

Tables touched: users, memberships.

Opening the Members tab fires `GET /api/members`, which `MemberService` filters down to the admin's own gym before the table renders. Editing a row sends `PUT /api/members/{id}`. Search and filter work as expected, editing and plan assignment both save correctly, and the member-side dashboard shows onboarding progress and enrollment status.

2.3 Subscription plans

Admins create, edit, activate, and deactivate plan tiers; members only ever see the active ones when they go to pay.

`PlanForm.jsx`, `PlanList.jsx`, and `PlanPicker.jsx` handle this on web; `PlanAdminAdapter` and `PlanAdapter` on Android. `PlanController` calls `PlanService`, which persists a `SubscriptionPlan` through `SubscriptionPlanRepository`.

Method	Endpoint	Description
GET	<code>/api/plans</code>	List plans (active only for members, all for admin)
POST	<code>/api/plans</code>	Create a plan
PUT	<code>/api/plans/{id}</code>	Update a plan

Table touched: `subscription_plans`.

One file worth calling out here is `MembershipStatusUtil.java`, which works out whether a given membership counts as Active, Expiring Soon, or Expired based on its end date every status badge in the app traces back to that one function.

Creating and editing a plan both work, deactivated plans disappear from the member's picker, and the status badges update correctly as a membership approaches its end date.

2.4 Online payments (PayMongo)

Members pay for a plan online, and the membership only activates once the payment actually confirms.

PlanPicker.jsx kicks this off, and PaymentHistory.jsx / PaymentSuccess/index.jsx handle the rest on web. On Android, checkout opens in a Custom Tab from DashboardViewModel1, reusing PlanAdapter and PaymentAdapter.

PaymentController calls into PaymentService, PayMongoService, and MembershipService together, backed by PaymentRepository and the Payment / PaymentStatus entities.

Method	Endpoint	Description
POST	/api/payments/checkout	Create a PayMongo checkout session
POST	/api/payments/webhook	PayMongo's payment-confirmed callback
POST	/api/payments/confirm-mock	Dev-only mock confirmation, used for testing
GET	/api/payments/me	Member's own payment history
GET	/api/payments	Admin view of all payments for the gym

Tables touched: payments, memberships.

A member picking a plan sends POST /api/payments/checkout, which creates a PENDING payment row and asks PayMongoService for a checkout session; the browser opens PayMongo's hosted page for GCash, Maya, or card. On success (through the real webhook, or the mock-confirm endpoint I use for local testing), PaymentService marks the row PAID and MembershipService.activateMembership() sets the start and end dates.

The mock checkout path activates a membership correctly, and the resulting payment shows up in both the member's Activity tab and the admin's Payments tab.

2.5 QR attendance tracking

Staff and admins scan a member's QR to check them in or out; members scan the gym's own QR once to enroll at that location.

QrScanner.jsx, GymQrCard.jsx, MemberQrCard.jsx, and AttendanceLogTable.jsx cover this on web. Android uses QrScannerDialogFragment for scanning and AttendanceLogAdapter for the log list.

QrController and AttendanceController both feed into QrAttendanceService, which reads/writes through AttendanceLogRepository and GymRepository.

Method	Endpoint	Description
GET	/api/qr/me	Member's personal QR code
GET	/api/qr/gym	Staff/admin gym-enrollment QR
POST	/api/attendance/scan	Staff scans a member QR (check-in / check-out)
POST	/api/attendance/scan-gym	Member scans the gym QR to enroll
GET	/api/attendance/me	Member's own attendance history
GET	/api/attendance/gym	Admin attendance log with filters

Tables touched: attendance_logs, users, gyms.

When the camera reads a QR payload, QrAttendanceService checks the membership's status first. If there's no open visit it creates a new attendance_logs row with a check-in time; if one's already open, it just fills in the check-out time on that same row.

Gym enrollment works, scanning correctly toggles check-in/check-out, expired memberships get denied at the scan step, and the attendance log can be filtered by date and by name.

2.6 Admin dashboard

This pulls the gym's KPIs and staff management into one place the admin's home tab, plus the other admin panels for plans, members, staff, attendance, and payments.

AdminDashboardPanel.jsx and KpiSummaryGrid.jsx build the web view; Android's DashboardFragment (admin sections) and KpiAdapter do the same job on mobile.

DashboardController and StaffController call DashboardService and StaffService, pulling from UserRepository, PaymentRepository, and AttendanceLogRepository.

Method	Endpoint	Description
GET	/api/dashboard/stats	KPI numbers for the admin home tab
GET	/api/staff	List staff accounts
PUT	/api/staff/{id}	Edit or deactivate a staff account

Tables touched: users, memberships, payments, attendance_logs.

Opening the Home tab fires GET /api/dashboard/stats, and DashboardService aggregates member counts, active vs. expired subscriptions, today's check-ins, and total payments into the numbers the KPI grid shows.

The KPI cards populate with live numbers, staff create/edit works from the Overview tab, and all the admin tabs load without errors on both web and mobile.

3. System Architecture

GymTrack is split into three layers. The web app and the Android app are two separate front ends, but they both talk to the same Spring Boot API, and that API is the only thing that touches the database directly neither client ever queries Postgres on its own.

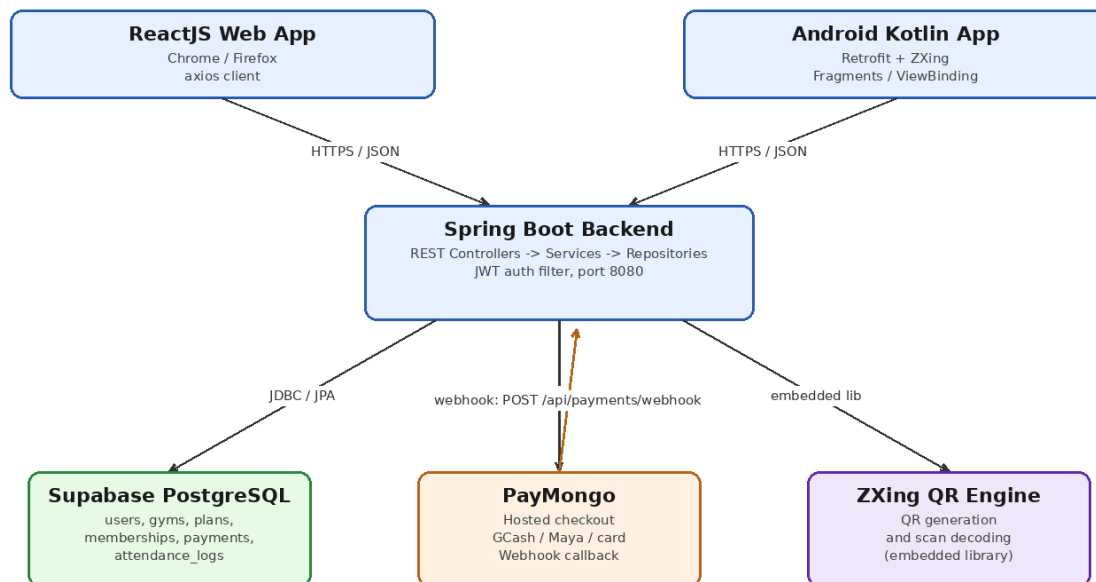


Fig. 1 - GymTrack three-tier architecture: web and mobile clients share one REST API, which persists to PostgreSQL and calls out to PayMongo for payments.

Request flow: a member paying for a plan

1. Member taps Subscribe on the Plans tab, web or Android.
2. Client sends POST /api/payments/checkout with the plan ID and JWT.
3. PaymentController hands off to PaymentService, which creates a PENDING payment row.
4. PayMongoService opens a checkout session and returns the URL.
5. Client opens PayMongo's hosted page (GCash, Maya, or card).
6. PayMongo posts to /api/payments/webhook once the payment succeeds.
7. PaymentService marks the payment PAID and MembershipService activates the membership.
8. The member's dashboard refreshes and shows Active with the new end date.

Request flow: a QR check-in

1. Staff opens the scanner on the web or mobile Scan tab.
2. The camera reads the member's QR payload (gymtrack:member:{userId}).
3. Client sends POST /api/attendance/scan with the QR data and JWT.
4. QrAttendanceService checks the membership status (Active / Expiring).
5. If there's no open visit, it creates an attendance_logs row with a check-in time.
6. If a visit is already open, it sets the check-out time on that same row.
7. The response shows the member's name, plan, status, and whether it was a check-in or check-out.